Software Project Pseudocodes

Hansraj Query Bridge

The Intelligent Query Resolution System

Submitted in partial fulfilment of the requirements of

DSC 15: Software Engineering

Prepared By:

Aryan Kushwaha 245703428

Moinak Roy 245703432

Gauri Tiwari 245703442

Yogita 245703448

Fida Zara 245703471

Aastha Garg 245703438

Use Case 1: Login / Authenticate

```
1: procedure  Authenticate(username, password)
2:     // 1. Initial Type and Null Validation
3:     if username is null or password is null
4:     then
5:         return error "Validation Failed: Credentials cannot be null"
6:     end if
7:     if typeof(username) != "string" or typeof(password) != "string"
8:     then
9:         return error "Validation Failed: Invalid data type"
10:    end if
11:    // 2. Boundary Validation
12:    if length(username) < 5 or length(username) > 50
13:    then
14:        return error "Validation Failed: Username length out of bounds"
15:    end if
16:    // 3. Character Sanitization Loop
17:    forbidden_chars ← array of characters {';', '-', "'", '"'}
18:    for i ← 0 to length(username) - 1
19:    do
20:        if username[i] in forbidden_chars
21:        then
22:            return error "Validation Failed: Illegal characters detected"
23:        end if
24:    end for
25:    // 4. Resilient External Call with Error Handling
26:    retry_count ← 0
27:    is_authenticated ← false
28:    auth_id ← null
29:    while retry_count < 3 and is_authenticated = false
30:    do
31:        try
32:            response ← call ExternalAuthService(username, password)
33:            if response.status_code = 200
34:            then
35:                is_authenticated ← true
36:                auth_id ← response.user_identity
37:            else
38:                retry_count ← retry_count + 1
39:            end if
40:        catch NetworkException
41:            retry_count ← retry_count + 1
42:        end try
43:    end while
44:    if is_authenticated = false
45:    then
46:        return error "Authentication Failed: Maximum attempts exceeded or service unavailable"
47:    end if
48:    // 5. Secure Parameterized Database Queries
```

```
49:   try
50:     query_user ← "SELECT user_id, role_id FROM USERS WHERE user_identity = ?"
51:     user_record ← executeParameterizedQuery(query_user, [auth_id])
52:     if user_record is empty
53:     then
54:       return error "Database Error: User verified externally but local account missing"
55:     end if
56:     query_role ← "SELECT role_name FROM ROLE WHERE role_id = ?"
57:     role_record ← executeParameterizedQuery(query_role, [user_record.role_id])
58:     if role_record is empty
59:     then
60:       return error "Database Error: User lacks assigned system role"
61:     end if
62:     // 6. Token Generation and Routing
63:     session_token ← generateSecureJWT(user_record.user_id, role_record.role_name, 3600)
64:     dashboard_url ← concat("/", role_record.role_name, "/dashboard")
65:     return success session_token, user_record.user_id, role_record.role_name, dashboard_url
66:   catch DatabaseException as db_err
67:     logError(db_err)
68:     return error "System Error: Unable to complete authentication transaction"
69:   end try
70: end procedure
```

Use Case 2: Ask AI Query

```
1: procedure ResolveQueryRAG(student_query, threshold_T)
2:   // 1. Initial Type, Null, and Boundary Validation
3:   if student_query is null or threshold_T is null
4:   then
5:     return error "Validation Failed: Inputs cannot be null"
6:   end if
7:   if typeof(student_query) != "string" or typeof(threshold_T) != "float"
8:   then
9:     return error "Validation Failed: Invalid data types"
10:  end if
11:  if length(student_query) < 5 or length(student_query) > 1000
12:  then
13:    return error "Validation Failed: Query length out of bounds"
14:  end if
15:  if threshold_T < 0.0 or threshold_T > 1.0
16:  then
17:    return error "Validation Failed: Threshold must be between 0.0 and 1.0"
18:  end if
19:  // 2. Generate Embedding with Network Resilience
20:  query_embedding ← null
21:  try
```

```
    query_embedding ← call GeminiAPI_GenerateEmbedding(student_query)
    if query_embedding is null or length(query_embedding) = 0
    then
        return error "API Error: Failed to generate valid embedding vector"
    end if
  catch NetworkException as net_err
    logError(net_err)
    return error "System Error: AI Embedding service unavailable"
  end try
  // 3. Secure Database Retrieval
  candidate_entries ← empty list
  try
    query_string ← "SELECT kb_id, content, source, embedding FROM D2_KNOWLEDGE
WHERE validity_status = ?"
    candidate_entries ← executeParameterizedQuery(query_string, ["Active"])
    if candidate_entries is empty
    then
        return response "Escalation Required", action "Raise Query to Admin", reason "Knowledge
Base empty"
    end if
  catch DatabaseException as db_err
    logError(db_err)
    return error "Database Error: Failed to retrieve Knowledge Base entries"
  end try
  // 4. Cosine Similarity Math
  results ← empty priority queue
  for each entry in candidate_entries
  do
    if entry.embedding is not null and length(entry.embedding) = length(query_embedding)
    then
        dot_product ← 0.0
        magnitude_q ← 0.0
        magnitude_e ← 0.0
        // Explicitly iterate through the vector arrays
        for i ← 0 to length(query_embedding) - 1
        do
            dot_product ← dot_product + (query_embedding[i] * entry.embedding[i])
            magnitude_q ← magnitude_q + (query_embedding[i] * query_embedding[i])
            magnitude_e ← magnitude_e + (entry.embedding[i] * entry.embedding[i])
        end for
        magnitude_q ← sqrt(magnitude_q)
        magnitude_e ← sqrt(magnitude_e)
        // Prevent division by zero
        if magnitude_q > 0 and magnitude_e > 0
        then
            similarity_score ← dot_product / (magnitude_q * magnitude_e)
            insert (entry, similarity_score) into results
        end if
    end if
  end for
```

```
70:   // 5. Evaluate Threshold and Extract Top K
71:   sort results descending by similarity_score
72:   top_k_context ← extract top 5 valid entries from results
73:   if length(top_k_context) = 0 or results[0].similarity_score < threshold_T
74:   then
75:      return response "Escalation Required", action "Raise Query to Admin"
76:   end if
77:   // 6. Generate Grounded AI Response
78:   try
79:     ai_answer ← call GeminiAPI_GenerateGroundedResponse(student_query, top_k_context)
80:      if ai_answer is null or length(ai_answer) = 0
81:      then
82:         return response "Escalation Required", action "Raise Query to Admin", reason "AI
Generation Failed"
83:      end if
84:      sources_list ← extract source fields from top_k_context
85:      return success "Grounded Answer", ai_answer, sources_list
86:   catch NetworkException as net_err
87:      logError(net_err)
88:      return error "System Error: AI Generation service unavailable"
89:   end try
90: end procedure
```

Use Case 3: Raise New Query

```
1: procedure  RaiseNewQuery(student_id, query_text, department_id, category)
2:   // 1. Initial Type, Null, and Boundary Validation
3:   if student_id is null or query_text is null or department_id is null or category is null
4:   then
5:      return error "Validation Failed: Inputs cannot be null"
6:   end if
7:   if typeof(query_text) != "string" or typeof(category) != "string"
8:   then
9:      return error "Validation Failed: Invalid data types"
10:  end if
11:  if length(query_text) < 10 or length(query_text) > 2000
12:  then
13:     return error "Validation Failed: Query text length out of bounds"
14:  end if
15:  // 2. Explicit Category Validation Loop
16:  valid_categories ← arrayofstrings {"Academic", "Administrative", "Technical", "Financial"}
17:  is_valid_category ← false
18:  for i ← 0 to length(valid_categories) - 1
19:  do
20:     if category = valid_categories[i]
21:     then
```

```
22:         is_valid_category ← true
23:         break
24:     end if
25:   end for
26:   if is_valid_category = false
27:   then
28:     return error "Validation Failed: Unrecognized query category"
29:   end if
30:   // 3. Relational Integrity Checks (Foreign Keys)
31:   try
32:     student_check ← "SELECT user_id FROM USERS WHERE user_id = ?"
33:     student_record ← executeParameterizedQuery(student_check, [student_id])
34:     if student_record is empty
35:     then
36:       return error "Database Error: Invalid student identity"
37:     end if
38:     dept_check ← "SELECT dept_id FROM DEPARTMENTS WHERE dept_id = ?"
39:     dept_record ← executeParameterizedQuery(dept_check, [department_id])
40:     if dept_record is empty
41:     then
42:       return error "Database Error: Invalid department reference"
43:     end if
44:   catch DatabaseException as db_err
45:     logError(db_err)
46:     return error "System Error: Unable to validate relational constraints"
47:   end try
48:   // 4. Secure Database Transaction
49:   new_query_id ← generateSecureUUID()
50:   current_time ← getSystemTimestamp()
51:   try
52:     // Begin Transaction to ensure both query and audit log succeed together
53:     beginTransaction()
54:     insert_query ← "INSERT INTO D3_QUERIES (query_id, student_id, department_id,
description, category, status, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
55:     executeParameterizedInsert(insert_query, [new_query_id, student_id, department_id,
query_text, category, "Open", current_time, current_time])
56:     insert_audit ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
57:     executeParameterizedInsert(insert_audit, [student_id, "Create", "Query", new_query_id,
current_time])
58:     commitTransaction()
59:   catch DatabaseException as db_err
60:     rollbackTransaction()
61:     logError(db_err)
62:     return error "Database Error: Failed to save new query"
63:   end try
64:   // 5. Fire-and-Forget Asynchronous Notification
65:   try
66:     call DispatchNotificationAsync("Admin", "New Query Escalated", new_query_id)
67:   catch NetworkException as net_err
```

```
68:       // Non-blocking error: log it but do not fail the query creation
69:       logError("Warning: Failed to dispatch notification for query " + new_query_id)
70:    end try
71:    return success "Query submitted successfully", new_query_id
72: end procedure
```

Use Case 4: Track Query Status

```
1: procedure  LoadStudentDashboard(student_id)
2:    // 1. Initial Type and Null Validation
3:    if student_id is null
4:    then
5:       return error "Validation Failed: Student ID cannot be null"
6:    end if
7:    if typeof(student_id) != "string"
8:    then
9:       return error "Validation Failed: Invalid data type for Student ID"
10:   end if
11:   // 2. Validate User Existence (Relational Integrity)
12:   try
13:      auth_check ← "SELECT user_id FROM USERS WHERE user_id = ?"
14:      auth_record ← executeParameterizedQuery(auth_check, [student_id])
15:      if auth_record is empty
16:      then
17:         return error "Database Error: Student identity not found"
18:      end if
19:   catch DatabaseException as db_err
20:      logError(db_err)
21:      return error "System Error: Unable to validate user identity"
22:   end try
23:   // 3. Retrieve Base Queries (Relying on DB Engine for optimized sorting)
24:   student_queries ← empty list
25:   try
26:      query_string ← "SELECT * FROM D3_QUERIES WHERE student_id = ? ORDER BY
updated_at DESC"
27:      student_queries ← executeParameterizedQuery(query_string, [student_id])
28:   catch DatabaseException as db_err
29:      logError(db_err)
30:      return error "Database Error: Failed to retrieve student queries"
31:   end try
32:   // If no queries exist, return the empty dashboard successfully
33:   if length(student_queries) = 0
34:   then
35:      return success "No queries found", empty list
36:   end if
37:   // 4. Data Aggregation Loop (Nested Retrievals)
```

```
38:    dashboard_view ← empty list
39:    for i ← 0 to length(student_queries) - 1
40:    do
41:        current_query ← student_queries[i]
42:        associated_docs ← empty list
43:        associated_meetings ← empty list
44:        // Wrap nested queries in a try-catch to prevent a single corrupted relation from crashing the whole dashboard
45:        try
46:            doc_sql ← "SELECT document_id, file_name, status FROM D4_DOCUMENTS WHERE query_id = ?"
47:            associated_docs ← executeParameterizedQuery(doc_sql, [current_query.query_id])
48:            meeting_sql ← "SELECT meeting_id, date, start_time, status FROM D5_MEETINGS WHERE query_id = ?"
49:            associated_meetings ← executeParameterizedQuery(meeting_sql, [current_query.query_id])
50:        catch DatabaseException as db_err
51:            logError("Warning: Failed to load relations for query " + current_query.query_id)
52:            associated_docs ← "Data unavailable"
53:            associated_meetings ← "Data unavailable"
54:        end try
55:        // 5. Construct Compiled Record
56:        compiled_record ← initialize new dictionary
57:        compiled_record["query_details"] ← current_query
58:        compiled_record["documents"] ← associated_docs
59:        compiled_record["meetings"] ← associated_meetings
60:        // Append to final view array
61:        insert compiled_record into dashboard_view at index i
62:    end for
63:    return success "Dashboard data retrieved successfully", dashboard_view
64: end procedure
```

Use Case 5: Upload Requested Documents

```
1: procedure  UploadDocument(student_id, query_id, file_obj)
2:    // 1. Initial Type and Null Validation
3:    if student_id is null or query_id is null or file_obj is null
4:    then
5:        return error "Validation Failed: Inputs cannot be null"
6:    end if
7:    if typeof(file_obj.name) != "string" or typeof(file_obj.size) != "integer"
8:    then
9:        return error "Validation Failed: Invalid file object structure"
10:   end if
11:   // 2. Explicit Physical File Constraints
12:   max_bytes ← 25 * 1024 * 1024  // 26,214,400 bytes (25MB)
13:   if file_obj.size = 0 or file_obj.size > max_bytes
```

```
14:  then
15:     return error "Validation Failed: File size must be between 1 byte and 25MB"
16:  end if
17:  valid_formats ← array of strings {"PDF", "JPG", "JPEG", "PNG"}
18:  file_extension ← toUpper(extractExtension(file_obj.name))
19:  is_valid_format ← false
20:  for i ← 0 to length(valid_formats) - 1
21:  do
22:     if file_extension = valid_formats[i]
23:     then
24:        is_valid_format ← true
25:        break
26:     end if
27:  end for
28:  if is_valid_format = false
29:  then
30:     return error "Validation Failed: Unsupported file format detected"
31:  end if
32:  // 3. Relational Integrity and Ownership Check
33:  try
34:     auth_check ← "SELECT student_id, status FROM D3_QUERIES WHERE query_id = ?"
35:     parent_query ← executeParameterizedQuery(auth_check, [query_id])
36:     if parent_query is empty
37:     then
38:        return error "Database Error: Invalid query reference"
39:     end if
40:     if parent_query.student_id != student_id
41:     then
42:        return error "Security Error: Unauthorized attempt to upload to another user's query"
43:     end if
44:  catch DatabaseException as db_err
45:     logError(db_err)
46:     return error "System Error: Unable to validate query ownership"
47:  end try
48:  // 4. External Cloud Storage Integration with Resilience
49:  storage_key ← null
50:  try
51:     storage_key ← call AWSS3_UploadStream(file_obj.stream, file_obj.name, query_id)
52:     if storage_key is null
53:     then
54:        return error "Cloud Error: S3 returned null storage key"
55:     end if
56:  catch NetworkException as net_err
57:     logError(net_err)
58:     return error "System Error: File upload service temporarily unavailable"
59:  end try
60:  // 5. Secure Database Transaction (Insert Document & Update Query)
61:  new_doc_id ← generateSecureUUID()
62:  current_time ← getSystemTimestamp()
63:  try
```

```
64:       beginTransaction()
65:       insert_doc ← "INSERT INTO D4_DOCUMENTS (document_id, query_id, uploaded_by,
file_name, file_type, file_size, storage_key, status, uploaded_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"
66:       executeParameterizedInsert(insert_doc, [new_doc_id, query_id, student_id, file_obj.name,
file_extension, file_obj.size, storage_key, "Pending Review", current_time])
67:       update_query ← "UPDATE D3_QUERIES SET status = ?, updated_at = ? WHERE query_id =
?"
68:       executeParameterizedUpdate(update_query, ["Document Submitted", current_time, query_id])
69:       commitTransaction()
70:    catch DatabaseException as db_err
71:       rollbackTransaction()
72:       logError(db_err)
73:       // Attempt to clean up orphaned S3 file if DB fails
74:       call AWSS3_Delete(storage_key)
75:       return error "Database Error: Failed to save document records"
76:    end try
77:    // 6. Fire-and-Forget Asynchronous Notification
78:    try
79:       call DispatchNotificationAsync("Staff", "Document Uploaded", new_doc_id)
80:    catch NetworkException as net_err
81:       logError("Warning: Failed to dispatch staff notification for document " + new_doc_id)
82:    end try
83:    return success "Document securely uploaded and linked", new_doc_id
84: end procedure
```

Use Case 6: View, Request Slot and Accept Meeting

```
1: procedure  ViewRequestSlotAndAcceptMeeting(user_id, action_type, slot_data)
2:    // 1. Initial Type and Null Validation
3:    if user_id is null or action_type is null or slot_data is null
4:    then
5:       return error "Validation Failed: Core parameters cannot be null"
6:    end if
7:    if typeof(user_id) != "string" or typeof(action_type) != "string" or typeof(slot_data) != "dictionary"
8:    then
9:       return error "Validation Failed: Invalid parameter data types"
10:   end if
11:   // 2. Explicit Action Type Validation Loop
12:   valid_actions ← array of strings {"ViewSlots", "RequestSlot", "AcceptMeeting"}
13:   is_valid_action ← false
14:   for i ← 0 to length(valid_actions) - 1
15:   do
16:      if action_type = valid_actions[i]
17:      then
18:         is_valid_action ← true
19:         break
```

```
20:     end if
21:   end for
22:   if is_valid_action = false
23:   then
24:     return error "Validation Failed: Action must be 'ViewSlots', 'RequestSlot', or 'AcceptMeeting'"
25:   end if
26:   // 3. Strict Role-Based Access Control (RBAC) Validation
27:   try
28:     rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
29:     role_record ← executeParameterizedQuery(rbac_query, [user_id])
30:     if role_record is empty
31:     then
32:       return error "Security Error: User account not found"
33:     end if
34:   catch DatabaseException as db_err
35:     logError(db_err)
36:     return error "System Error: Unable to validate permissions"
37:   end try
38:   current_time ← getSystemTimestamp()
39:   reference_id ← null
40:   target_recipient_id ← null
41:   notification_msg ← ""
42:   // 4. Branch A: View Available Slots (Student or TIC)
43:   if action_type = "ViewSlots"
44:   then
45:     if slot_data["tic_id"] is null or slot_data["target_date"] is null
46:     then
47:       return error "Validation Failed: TIC ID and target date required to view slots"
48:     end if
49:     try
50:       slot_query ← "SELECT slot_id, start_time, end_time, status FROM D6_TIME_SLOTS
WHERE assigned_tic = ? AND slot_date = ? AND status = ?"
51:       available_slots ← executeParameterizedQuery(slot_query, [slot_data["tic_id"],
slot_data["target_date"], "Available"])
52:       return success "Available slots retrieved successfully", available_slots
53:     catch DatabaseException as db_err
54:       logError(db_err)
55:       return error "Database Error: Failed to retrieve slot availability"
56:     end try
57:   // 5. Branch B: Request Slot (Student Action)
58:   else if action_type = "RequestSlot"
59:   then
60:     if role_record.role_name != "Student"
61:     then
62:       return error "Security Error: Only students can request meeting slots"
63:     end if
64:     if slot_data["slot_id"] is null or slot_data["query_id"] is null
65:     then
66:       return error "Validation Failed: Slot ID and Query ID required for request"
```

```
67:     end if
68:     try
69:         // Verify Slot Exists and is Open
70:         slot_check ← "SELECT assigned_tic, status FROM D6_TIME_SLOTS WHERE slot_id = ?"
71:         target_slot ← executeParameterizedQuery(slot_check, [slot_data["slot_id"]])
72:         if target_slot is empty or target_slot.status != "Available"
73:         then
74:             return error "State Error: Selected slot is no longer available"
75:         end if
76:         target_recipient_id ← target_slot.assigned_tic
77:         reference_id ← slot_data["slot_id"]
78:         beginTransaction()
79:         update_slot ← "UPDATE D6_TIME_SLOTS SET status = ?, student_id = ?, updated_at = ?
WHERE slot_id = ?"
80:         executeParameterizedUpdate(update_slot, ["Requested", user_id, current_time, reference_id])
81:         insert_audit ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
82:         executeParameterizedInsert(insert_audit, [user_id, "Request Slot", "Slot", reference_id,
current_time])
83:         commitTransaction()
84:         notification_msg ← "Meeting Slot Requested"
85:     catch DatabaseException as db_err
86:         rollbackTransaction()
87:         logError(db_err)
88:         return error "Database Error: Failed to process slot request securely"
89:     end try
90:  // 6. Branch C: Accept Meeting (TIC Action)
91:  else if action_type = "AcceptMeeting"
92:  then
93:     if role_record.role_name != "TIC"
94:     then
95:         return error "Security Error: Only TICs can accept meeting slots"
96:     end if
97:     if slot_data["slot_id"] is null
98:     then
99:         return error "Validation Failed: Slot ID required to accept meeting"
100:    end if
101:    try
102:        // Verify Slot Ownership and Requested State (IDOR Prevention)
103:        slot_verify ← "SELECT assigned_tic, student_id, status FROM D6_TIME_SLOTS WHERE
slot_id = ?"
104:        target_slot ← executeParameterizedQuery(slot_verify, [slot_data["slot_id"]])
105:        if target_slot is empty
106:        then
107:            return error "Database Error: Slot reference not found"
108:        end if
109:        if target_slot.assigned_tic != user_id
110:        then
111:            return error "Security Error: Unauthorized attempt to accept another TIC's slot"
112:        end if
```

```
113:         if target_slot.status != "Requested"
114:         then
115:             return error "State Error: Slot must be in 'Requested' state to accept"
116:         end if
117:         target_recipient_id ← target_slot.student_id
118:         reference_id ← slot_data["slot_id"]
119:         beginTransaction()
120:         accept_update ← "UPDATE D6_TIME_SLOTS SET status = ?, updated_at = ? WHERE
slot_id = ?"
121:         executeParameterizedUpdate(accept_update, ["Accepted", current_time, reference_id])
122:         insert_audit ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
123:         executeParameterizedInsert(insert_audit, [user_id, "Accept Meeting", "Slot", reference_id,
current_time])
124:         commitTransaction()
125:         notification_msg ← "Meeting Accepted"
126:     catch DatabaseException as db_err
127:         rollbackTransaction()
128:         logError(db_err)
129:         return error "Database Error: Failed to accept meeting slot safely"
130:     end try
131:  end if
132:  // 7. Fire-and-Forget Asynchronous Notification
133:  if action_type != "ViewSlots"
134:  then
135:     try
136:         call DispatchNotificationAsync(target_recipient_id, notification_msg, reference_id)
137:     catch NetworkException as net_err
138:         logError("Warning: Failed to dispatch notification for slot " + reference_id)
139:     end try
140:  end if
141:  return success concat("Action successfully processed: ", action_type), reference_id
142: end procedure
```

Use Case 7: Receive Notifications

```
1: procedure  DispatchNotificationAsync(target, event_type, reference_id)
2:     // 1. Initial Type and Null Validation
3:     if target is null or event_type is null or reference_id is null
4:     then
5:         return error "Validation Failed: Inputs cannot be null"
6:     end if
7:     if typeof(target) != "string" or typeof(event_type) != "string"
8:     then
9:         return error "Validation Failed: Invalid data types"
10:    end if
```

```
11:   // 2. Validate Event Type
12:   valid_events ← array of strings {"New Query Escalated", "Query Assigned", "Query Resolved",
"Document Required", "Document Uploaded", "Document Accepted", "Document Rejected", "Meeting
Scheduled", "Meeting Cancelled"}
13:   is_valid_event ← false
14:   for i ← 0 to length(valid_events) - 1
15:   do
16:      if event_type = valid_events[i]
17:      then
18:         is_valid_event ← true
19:         break
20:      end if
21:   end for
22:   if is_valid_event = false
23:   then
24:      return error "Validation Failed: Unrecognized event type"
25:   end if
26:   // 3. Resolve Target to Email Addresses
27:   recipient_emails ← empty list
28:   try
29:      // Check if target is a Role
30:      if target = "Admin" or target = "TIC" or target = "Staff"
31:      then
32:         role_query ← "SELECT U.email FROM USERS U JOIN ROLE R ON U.role_id = R.role_id
WHERE R.role_name = ?"
33:         // If Staff, we need both Admin and TIC
34:         if target = "Staff"
35:         then
36:            role_query ← "SELECT U.email FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE R.role_name IN ('Admin', 'TIC')"
37:            role_records ← executeParameterizedQuery(role_query, [])
38:         else
39:            role_records ← executeParameterizedQuery(role_query, [target])
40:         end if
41:         for each record in role_records
42:         do
43:            append record.email to recipient_emails
44:         end for
45:      // Otherwise, assume target is a specific User ID
46:      else
47:         user_query ← "SELECT email FROM USERS WHERE user_id = ?"
48:         user_record ← executeParameterizedQuery(user_query, [target])
49:         if user_record is not empty
50:         then
51:            append user_record.email to recipient_emails
52:         end if
53:      end if
54:   catch DatabaseException as db_err
55:      logError(db_err)
56:      return error "Database Error: Failed to resolve target email addresses"
```

```
    end try
    if length(recipient_emails) = 0
    then
        return error "Resolution Error: No valid email addresses found for target"
    end if
    // 4. Construct Email Content
    email_subject ← concat("System Notification: ", event_type)
    email_body ← concat("An event '", event_type, "' has occurred concerning reference ID: ",
reference_id, ". Please log in to your dashboard to view details.")
    // 5. Dispatch Loop with Individual Error Handling
    success_count ← 0
    failure_count ← 0
    current_time ← getSystemTimestamp()
    for each email_address in recipient_emails
    do
        try
            // Simulating external email provider call (e.g., Resend/SendGrid)
            api_response ← call ExternalEmailService(email_address, email_subject, email_body)
            if api_response.status_code = 200
            then
                success_count ← success_count + 1
                // Log successful dispatch
                try
                    log_query ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity,
entity_id, timestamp) VALUES (?, ?, ?, ?, ?)"
                    executeParameterizedInsert(log_query, ["System", concat("Email Sent to ",
email_address), "Notification", reference_id, current_time])
                catch DatabaseException as db_log_err
                    logError("Warning: Failed to log notification success for " + email_address)
                end try
            else
                failure_count ← failure_count + 1
                logError("Email API Error: Failed to send to " + email_address + " with status " +
api_response.status_code)
            end if
        catch NetworkException as net_err
            failure_count ← failure_count + 1
            logError("Network Error: Could not reach email service for " + email_address)
        end try
    end for
    if success_count > 0
    then
        return success concat("Dispatched ", success_count, " notifications. Failures: ", failure_count)
    else
        return error "Dispatch Failed: Unable to send any notifications"
    end if
end procedure
```

Use Case 8: View New Queries

```
1: procedure  AdminViewNewQueries(admin_id)
2:     // 1. Initial Type and Null Validation
3:     if admin_id is null
4:     then
5:         return error "Validation Failed: Admin ID cannot be null"
6:     end if
7:     if typeof(admin_id) != "string"
8:     then
9:         return error "Validation Failed: Invalid data type for Admin ID"
10:    end if
11:    // 2. Strict Role-Based Access Control (RBAC) Validation
12:    try
13:        rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
14:        role_record ← executeParameterizedQuery(rbac_query, [admin_id])
15:        if role_record is empty
16:        then
17:            return error "Database Error: Admin identity not found"
18:        end if
19:        if role_record.role_name != "Admin"
20:        then
21:            return error "Security Error: Unauthorized access. Admin privileges required."
22:        end if
23:    catch DatabaseException as db_err
24:        logError(db_err)
25:        return error "System Error: Unable to validate permissions"
26:    end try
27:    // 3. Secure Base Query Retrieval
28:    open_queries ← empty list
29:    try
30:        // Fetch only queries with "Open" status, sorted chronologically
31:        query_string ← "SELECT * FROM D3_QUERIES WHERE status = ? ORDER BY created_at
ASC"
32:        open_queries ← executeParameterizedQuery(query_string, ["Open"])
33:    catch DatabaseException as db_err
34:        logError(db_err)
35:        return error "Database Error: Failed to retrieve open queries"
36:    end try
37:    // If no open queries exist, return gracefully
38:    if length(open_queries) = 0
39:    then
40:        return success "No new queries found", empty list
41:    end if
42:    // 4. Data Aggregation Loop for Dashboard Context
43:    admin_dashboard_view ← empty list
44:    for i ← 0 to length(open_queries) - 1
45:    do
46:        current_query ← open_queries[i]
```

```
47:       student_details ← initialize new dictionary
48:       // Resolve student details within an isolated try-catch to prevent dashboard failure
49:       try
50:          student_sql ← "SELECT name, email FROM USERS WHERE user_id = ?"
51:          student_record ← executeParameterizedQuery(student_sql, [current_query.student_id])
52:          if student_record is not empty
53:          then
54:             student_details["name"] ← student_record.name
55:             student_details["email"] ← student_record.email
56:          else
57:             student_details["name"] ← "Unknown Student"
58:             student_details["email"] ← "Unknown Email"
59:          end if
60:       catch DatabaseException as db_err
61:          logError("Warning: Failed to load student details for query " + current_query.query_id)
62:          student_details["name"] ← "Data unavailable"
63:          student_details["email"] ← "Data unavailable"
64:       end try
65:       // 5. Construct Compiled Record
66:       compiled_record ← initialize new dictionary
67:       compiled_record["query_details"] ← current_query
68:       compiled_record["student_info"] ← student_details
69:       // Append to final view array
70:       insert compiled_record into admin_dashboard_view at index i
71:    end for
72:    return success "Admin dashboard data retrieved successfully", admin_dashboard_view
73: end procedure
```

Use Case 9: Monitor and Resolve Queries directly

```
1: procedure  AdminResolveQuery(admin_id, query_id, resolution_text, add_to_kb_flag)
2:    // 1. Initial Type, Null, and Boundary Validation
3:    if admin_id is null or query_id is null or resolution_text is null or add_to_kb_flag is null
4:    then
5:       return error "Validation Failed: Parameters cannot be null"
6:    end if
7:    if typeof(admin_id) != "string" or typeof(query_id) != "string" or typeof(resolution_text) != "string"
or typeof(add_to_kb_flag) != "boolean"
8:    then
9:       return error "Validation Failed: Invalid parameter data types"
10:   end if
11:   if length(resolution_text) < 10 or length(resolution_text) > 5000
12:   then
13:      return error "Validation Failed: Resolution text length out of bounds"
14:   end if
15:   // 2. Strict Role-Based Access Control (RBAC) Validation
```

```
    try
        rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
        role_record ← executeParameterizedQuery(rbac_query, [admin_id])
        if role_record is empty or role_record.role_name != "Admin"
        then
            return error "Security Error: Unauthorized access. Admin privileges required."
        end if
    catch DatabaseException as db_err
        logError(db_err)
        return error "System Error: Unable to validate permissions"
    end try
    // 3. Relational Integrity: Validate Query Existence
    try
        query_check ← "SELECT student_id, description, category FROM D3_QUERIES WHERE
query_id = ?"
        target_query ← executeParameterizedQuery(query_check, [query_id])
        if target_query is empty
        then
            return error "Database Error: Target query reference not found"
        end if
    catch DatabaseException as db_err
        logError(db_err)
        return error "System Error: Unable to validate query reference"
    end try
    // 4. Secure Database Transaction for Resolution and Audit
    current_time ← getSystemTimestamp()
    try
        beginTransaction()
        update_sql ← "UPDATE D3_QUERIES SET status = ?, resolution = ?, resolved_by = ?,
updated_at = ? WHERE query_id = ?"
        executeParameterizedUpdate(update_sql, ["Resolved", resolution_text, admin_id, current_time,
query_id])
        audit_sql ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
        executeParameterizedInsert(audit_sql, [admin_id, "Resolve", "Query", query_id, current_time])
        commitTransaction()
    catch DatabaseException as db_err
        rollbackTransaction()
        logError(db_err)
        return error "Database Error: Failed to resolve query and commit audit log"
    end try
    // 5. Conditional Knowledge Base Escalation (Isolated from main transaction)
    if add_to_kb_flag = true
    then
        try
            kb_payload ← initialize new dictionary
            kb_payload["title"] ← target_query.description
            kb_payload["content"] ← resolution_text
            kb_payload["category"] ← target_query.category
```

```
61:         kb_payload["source"] ← "Admin Resolution"
62:         // Call procedure  6.0
63:         call KnowledgeBaseManagement(admin_id, "Create", kb_payload)
64:       catch SystemException as sys_err
65:         // Log failure but do not fail the overall query resolution
66:         logError("Warning: Failed to add resolution to Knowledge Base for query " + query_id)
67:       end try
68:     end if
69:     // 6. Fire-and-Forget Asynchronous Notification
70:     try
71:       call DispatchNotificationAsync(target_query.student_id, "Query Resolved", query_id)
72:     catch NetworkException as net_err
73:       logError("Warning: Failed to dispatch resolution notification to student for query " + query_id)
74:     end try
75:     return success "Query successfully resolved", query_id
76: end procedure
```

Use Case 10: Forward Query to TIC

```
1: procedure  ForwardQueryToTIC(admin_id, query_id, tic_id)
2:     // 1. Initial Type and Null Validation
3:     if admin_id is null or query_id is null or tic_id is null
4:     then
5:       return error "Validation Failed: Parameters cannot be null"
6:     end if
7:     if typeof(admin_id) != "string" or typeof(query_id) != "string" or typeof(tic_id) != "string"
8:     then
9:       return error "Validation Failed: Invalid parameter data types"
10:    end if
11:    // 2. Caller RBAC Validation (Must be Admin)
12:    try
13:       admin_rbac ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
14:       admin_record ← executeParameterizedQuery(admin_rbac, [admin_id])
15:       if admin_record is empty or admin_record.role_name != "Admin"
16:       then
17:         return error "Security Error: Unauthorized access. Admin privileges required."
18:       end if
19:    catch DatabaseException as db_err
20:       logError(db_err)
21:       return error "System Error: Unable to validate caller permissions"
22:    end try
23:    // 3. Target User Validation (Crucial Security Check: Must be TIC)
24:    try
25:       tic_rbac ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id = R.role_id
WHERE U.user_id = ?"
```

```
26:       tic_record ← executeParameterizedQuery(tic_rbac, [tic_id])
27:       if tic_record is empty
28:       then
29:          return error "Database Error: Target user identity not found"
30:       end if
31:       if tic_record.role_name != "TIC"
32:       then
33:          return error "Security Error: Cannot assign query. Target user is not a Teacher-in-Charge."
34:       end if
35:    catch DatabaseException as db_err
36:       logError(db_err)
37:       return error "System Error: Unable to validate target user role"
38:    end try
39:    // 4. Relational Integrity: Validate Query Existence
40:    try
41:       query_check ← "SELECT status FROM D3_QUERIES WHERE query_id = ?"
42:       target_query ← executeParameterizedQuery(query_check, [query_id])
43:       if target_query is empty
44:       then
45:          return error "Database Error: Target query reference not found"
46:       end if
47:    catch DatabaseException as db_err
48:       logError(db_err)
49:       return error "System Error: Unable to validate query reference"
50:    end try
51:    // 5. Secure Database Transaction for Assignment and Audit
52:    current_time ← getSystemTimestamp()
53:    try
54:       beginTransaction()
55:       update_sql ← "UPDATE D3_QUERIES SET assigned_tic = ?, status = ?, updated_at = ?
WHERE query_id = ?"
56:       executeParameterizedUpdate(update_sql, [tic_id, "Assigned to TIC", current_time, query_id])
57:       audit_sql ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
58:       executeParameterizedInsert(audit_sql, [admin_id, "Forward to TIC", "Query", query_id,
current_time])
59:       commitTransaction()
60:    catch DatabaseException as db_err
61:       rollbackTransaction()
62:       logError(db_err)
63:       return error "Database Error: Failed to assign query and commit audit log"
64:    end try
65:    // 6. Fire-and-Forget Asynchronous Notification
66:    try
67:       call DispatchNotificationAsync(tic_id, "Query Assigned", query_id)
68:    catch NetworkException as net_err
69:       logError("Warning: Failed to dispatch assignment notification to TIC for query " + query_id)
70:    end try
71:    return success "Query successfully forwarded to TIC", query_id
72: end procedure
```

Use Case 11: Manage and Maintain Knowledge Base

```
1: procedure KnowledgeBaseManagement(admin_id, action_type, entry_data)
2:     // 1. Initial Type and Null Validation
3:     if admin_id is null or action_type is null or entry_data is null
4:     then
5:         return error "Validation Failed: Base parameters cannot be null"
6:     end if
7:     if typeof(action_type) != "string" or typeof(entry_data) != "dictionary"
8:     then
9:         return error "Validation Failed: Invalid parameter data types"
10:    end if
11:    // 2. Explicit Action Type Validation
12:    valid_actions ← array of strings {"Create", "Update", "Deactivate"}
13:    is_valid_action ← false
14:    for i ← 0 to length(valid_actions) - 1
15:    do
16:        if action_type = valid_actions[i]
17:        then
18:            is_valid_action ← true
19:            break
20:        end if
21:    end for
22:    if is_valid_action = false
23:    then
24:        return error "Validation Failed: Unrecognized action type"
25:    end if
26:    // 3. Strict Role-Based Access Control (RBAC)
27:    try
28:        rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
29:        role_record ← executeParameterizedQuery(rbac_query, [admin_id])
30:        if role_record is empty
31:        then
32:            return error "Database Error: User identity not found"
33:        end if
34:        if role_record.role_name != "Admin" and role_record.role_name != "Authorized TIC"
35:        then
36:            return error "Security Error: Unauthorized modification attempt"
37:        end if
38:    catch DatabaseException as db_err
39:        logError(db_err)
40:        return error "System Error: Unable to validate permissions"
41:    end try
42:    current_time ← getSystemTimestamp()
43:    // 4. Branch: Create Entry
44:    if action_type = "Create"
45:    then
46:        if entry_data["title"] is null or entry_data["content"] is null
47:        then
48:            return error "Validation Failed: Title and content are required for creation"
```

```
49:     end if
50:     embedding_vector ← null
51:     try
52:         embedding_vector ← call GeminiAPI_GenerateEmbedding(entry_data["content"])
53:         if embedding_vector is null or length(embedding_vector) = 0
54:         then
55:             return error "API Error: Failed to generate vector embedding"
56:         end if
57:     catch NetworkException as net_err
58:         logError(net_err)
59:         return error "System Error: AI Embedding service unavailable"
60:     end try
61:     new_kb_id ← generateSecureUUID()
62:     try
63:         insert_kb ← "INSERT INTO D2_KNOWLEDGE (kb_id, title, content, category, source,
verification_status, validity_status, embedding, created_by, created_at, updated_at) VALUES (?, ?, ?,
?, ?, ?, ?, ?, ?, ?, ?, ?)"
64:         executeParameterizedInsert(insert_kb, [new_kb_id, entry_data["title"], entry_data["content"],
entry_data["category"], entry_data["source"], "Verified", "Active", embedding_vector, admin_id,
current_time, current_time])
65:         return success "Knowledge Base entry created", new_kb_id
66:     catch DatabaseException as db_err
67:         logError(db_err)
68:         return error "Database Error: Failed to insert new KB entry"
69:     end try
70: // 5. Branch: Update Entry
71: else if action_type = "Update"
72: then
73:     if entry_data["kb_id"] is null
74:     then
75:         return error "Validation Failed: KB ID required for update"
76:     end if
77:     try
78:         fetch_kb ← "SELECT content FROM D2_KNOWLEDGE WHERE kb_id = ?"
79:         existing_entry ← executeParameterizedQuery(fetch_kb, [entry_data["kb_id"]])
80:         if existing_entry is empty
81:         then
82:             return error "Database Error: Knowledge Base entry not found"
83:         end if
84:         update_query ← "UPDATE D2_KNOWLEDGE SET title = ?, category = ?, source = ?,
updated_at = ?"
85:         update_params ← array of variables {entry_data["title"], entry_data["category"],
entry_data["source"], current_time}
86:         // Conditionally regenerate embeddings only if content changed
87:         if entry_data["content"] != null and entry_data["content"] != existing_entry.content
88:         then
89:             new_embedding ← call GeminiAPI_GenerateEmbedding(entry_data["content"])
90:             update_query ← concat(update_query, ", content = ?, embedding = ?")
91:             append entry_data["content"] to update_params
92:             append new_embedding to update_params
```

```
93:          end if
94:          update_query ← concat(update_query, " WHERE kb_id = ?")
95:          append entry_data["kb_id"] to update_params
96:          executeParameterizedUpdate(update_query, update_params)
97:          return success "Knowledge Base entry updated", entry_data["kb_id"]
98:       catch DatabaseException as db_err
99:          logError(db_err)
100:         return error "Database Error: Failed to update KB entry"
101:      catch NetworkException as net_err
102:         logError(net_err)
103:         return error "System Error: AI Embedding service unavailable during update"
104:      end try
105:   // 6. Branch: Deactivate Entry
106:   else if action_type = "Deactivate"
107:   then
108:      if entry_data["kb_id"] is null
109:      then
110:         return error "Validation Failed: KB ID required for deactivation"
111:      end if
112:      try
113:         deact_query ← "UPDATE D2_KNOWLEDGE SET validity_status = ?, updated_at = ?
WHERE kb_id = ?"
114:         rows_affected ← executeParameterizedUpdate(deact_query, ["Inactive", current_time,
entry_data["kb_id"]])
115:         if rows_affected = 0
116:         then
117:            return error "Database Error: Target entry not found or already inactive"
118:         end if
119:         return success "Knowledge Base entry deactivated", entry_data["kb_id"]
120:      catch DatabaseException as db_err
121:         logError(db_err)
122:         return error "Database Error: Failed to deactivate KB entry"
123:      end try
124:   end if
125: end procedure
```

Use Case 12: View Assigned Queries

```
1: procedure TICViewAssignedQueries(tic_id)
2:    // 1. Initial Type and Null Validation
3:    if tic_id is null
4:    then
5:       return error "Validation Failed: TIC ID cannot be null"
6:    end if
7:    if typeof(tic_id) != "string"
8:    then
```

```
9:        return error "Validation Failed: Invalid data type for TIC ID"
10:   end if
11:   // 2. Strict Role-Based Access Control (RBAC) Validation
12:   try
13:       rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
14:       role_record ← executeParameterizedQuery(rbac_query, [tic_id])
15:       if role_record is empty
16:       then
17:           return error "Database Error: User identity not found"
18:       end if
19:       if role_record.role_name != "TIC"
20:       then
21:           return error "Security Error: Unauthorized access. TIC privileges required."
22:       end if
23:   catch DatabaseException as db_err
24:       logError(db_err)
25:       return error "System Error: Unable to validate permissions"
26:   end try
27:   // 3. Secure Base Query Retrieval (Filtered by Ownership)
28:   assigned_queries ← empty list
29:   try
30:       // Fetch only queries assigned to this specific TIC, sorted chronologically
31:       query_string ← "SELECT * FROM D3_QUERIES WHERE assigned_tic = ? ORDER BY
updated_at DESC"
32:       assigned_queries ← executeParameterizedQuery(query_string, [tic_id])
33:   catch DatabaseException as db_err
34:       logError(db_err)
35:       return error "Database Error: Failed to retrieve assigned queries"
36:   end try
37:   // If no queries are assigned to this TIC, return gracefully
38:   if length(assigned_queries) = 0
39:   then
40:       return success "No assigned queries found", empty list
41:   end if
42:   // 4. Data Aggregation Loop for Dashboard Context
43:   tic_dashboard_view ← empty list
44:   for i ← 0 to length(assigned_queries) - 1
45:   do
46:       current_query ← assigned_queries[i]
47:       student_details ← initialize new dictionary
48:       // Resolve student details within an isolated try-catch to prevent dashboard failure
49:       try
50:           student_sql ← "SELECT name, email, department_id FROM USERS WHERE user_id = ?"
51:           student_record ← executeParameterizedQuery(student_sql, [current_query.student_id])
52:           if student_record is not empty
53:           then
54:               student_details["name"] ← student_record.name
55:               student_details["email"] ← student_record.email
56:               student_details["department_id"] ← student_record.department_id
```

```
57:          else
58:              student_details["name"] ← "Unknown Student"
59:              student_details["email"] ← "Unknown Email"
60:              student_details["department_id"] ← "Unknown"
61:          end if
62:      catch DatabaseException as db_err
63:          logError("Warning: Failed to load student details for query " + current_query.query_id)
64:          student_details["name"] ← "Data unavailable"
65:          student_details["email"] ← "Data unavailable"
66:          student_details["department_id"] ← "Data unavailable"
67:      end try
68:      // 5. Construct Compiled Record
69:      compiled_record ← initialize new dictionary
70:      compiled_record["query_details"] ← current_query
71:      compiled_record["student_info"] ← student_details
72:      // Append to final view array
73:      insert compiled_record into tic_dashboard_view at index i
74:  end for
75:  return success "TIC dashboard data retrieved successfully", tic_dashboard_view
76: end procedure
```

Use Case 13: Review and Resolve departmental queries

```
1: procedure  TICResolveQuery(tic_id, query_id, resolution_text, add_to_kb_flag)
2:     // 1. Initial Type, Null, and Boundary Validation
3:     if tic_id is null or query_id is null or resolution_text is null or add_to_kb_flag is null
4:     then
5:         return error "Validation Failed: Parameters cannot be null"
6:     end if
7:     if typeof(tic_id) != "string" or typeof(query_id) != "string" or typeof(resolution_text) != "string" or
typeof(add_to_kb_flag) != "boolean"
8:     then
9:         return error "Validation Failed: Invalid parameter data types"
10:    end if
11:    if length(resolution_text) < 10 or length(resolution_text) > 5000
12:    then
13:        return error "Validation Failed: Resolution text length out of bounds"
14:    end if
15:    // 2. Strict Role-Based Access Control (RBAC) Validation
16:    try
17:        rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
18:        role_record ← executeParameterizedQuery(rbac_query, [tic_id])
19:        if role_record is empty or role_record.role_name != "TIC"
20:        then
21:            return error "Security Error: Unauthorized access. TIC privileges required."
```

```
22:       end if
23:    catch DatabaseException as db_err
24:       logError(db_err)
25:       return error "System Error: Unable to validate permissions"
26:    end try
27:    // 3. Explicit Ownership and Integrity Validation (IDOR Prevention)
28:    try
29:       query_check ← "SELECT assigned_tic, student_id, description, category FROM D3_QUERIES
WHERE query_id = ?"
30:       target_query ← executeParameterizedQuery(query_check, [query_id])
31:       if target_query is empty
32:       then
33:          return error "Database Error: Target query reference not found"
34:       end if
35:       // Crucial check: Ensure the TIC attempting to resolve actually owns the ticket
36:       if target_query.assigned_tic != tic_id
37:       then
38:          return error "Security Error: Unauthorized. You can only resolve queries explicitly assigned
to you."
39:       end if
40:    catch DatabaseException as db_err
41:       logError(db_err)
42:       return error "System Error: Unable to validate query ownership"
43:    end try
44:    // 4. Secure Database Transaction for Resolution and Audit
45:    current_time ← getSystemTimestamp()
46:    try
47:       beginTransaction()
48:       update_sql ← "UPDATE D3_QUERIES SET status = ?, resolution = ?, resolved_by = ?,
updated_at = ? WHERE query_id = ?"
49:       executeParameterizedUpdate(update_sql, ["Resolved", resolution_text, tic_id, current_time,
query_id])
50:       audit_sql ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
51:       executeParameterizedInsert(audit_sql, [tic_id, "Resolve", "Query", query_id, current_time])
52:       commitTransaction()
53:    catch DatabaseException as db_err
54:       rollbackTransaction()
55:       logError(db_err)
56:       return error "Database Error: Failed to resolve query and commit audit log"
57:    end try
58:    // 5. Conditional Knowledge Base Escalation
59:    if add_to_kb_flag = true
60:    then
61:       try
62:          kb_payload ← initialize new dictionary
63:          kb_payload["title"] ← target_query.description
64:          kb_payload["content"] ← resolution_text
65:          kb_payload["category"] ← target_query.category
66:          kb_payload["source"] ← concat("TIC Resolution - ", tic_id)
```

```
67:          // Call procedure  6.0 seamlessly
68:          call KnowledgeBaseManagement(tic_id, "Create", kb_payload)
69:       catch SystemException as sys_err
70:          // Log failure but do not fail the overall query resolution
71:          logError("Warning: Failed to add resolution to Knowledge Base for query " + query_id)
72:       end try
73:    end if
74:    // 6. Fire-and-Forget Asynchronous Notification
75:    try
76:       call DispatchNotificationAsync(target_query.student_id, "Query Resolved", query_id)
77:    catch NetworkException as net_err
78:       logError("Warning: Failed to dispatch resolution notification to student for query " + query_id)
79:    end try
80:    return success "Query successfully resolved", query_id
81: end procedure
```

Use Case 14: Request Student Documents

```
1: procedure  TICRequestDocument(tic_id, query_id, doc_note)
2:    // 1. Initial Type, Null, and Boundary Validation
3:    if tic_id is null or query_id is null or doc_note is null
4:    then
5:       return error "Validation Failed: Parameters cannot be null"
6:    end if
7:    if typeof(tic_id) != "string" or typeof(query_id) != "string" or typeof(doc_note) != "string"
8:    then
9:       return error "Validation Failed: Invalid parameter data types"
10:   end if
11:   if length(doc_note) < 5 or length(doc_note) > 1000
12:   then
13:      return error "Validation Failed: Document request note length out of bounds"
14:   end if
15:   // 2. Strict Role-Based Access Control (RBAC) Validation
16:   try
17:      rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
18:      role_record ← executeParameterizedQuery(rbac_query, [tic_id])
19:      if role_record is empty or role_record.role_name != "TIC"
20:      then
21:         return error "Security Error: Unauthorized access. TIC privileges required."
22:      end if
23:   catch DatabaseException as db_err
24:      logError(db_err)
25:      return error "System Error: Unable to validate permissions"
26:   end try
27:   // 3. Explicit Ownership and State Validation (IDOR Prevention)
```

```
    try
        query_check ← "SELECT assigned_tic, student_id, status FROM D3_QUERIES WHERE query_id = ?"
        target_query ← executeParameterizedQuery(query_check, [query_id])
        if target_query is empty
        then
            return error "Database Error: Target query reference not found"
        end if
        // Ensure the TIC attempting to request documents actually owns the ticket
        if target_query.assigned_tic != tic_id
        then
            return error "Security Error: Unauthorized. You can only manage queries explicitly assigned to you."
        end if
        // Prevent requesting documents on closed queries
        if target_query.status = "Resolved" or target_query.status = "Closed"
        then
            return error "State Error: Cannot request documents for a closed or resolved query."
        end if
    catch DatabaseException as db_err
        logError(db_err)
        return error "System Error: Unable to validate query ownership and state"
    end try
    // 4. Secure Database Transaction for State Transition and Audit
    current_time ← getSystemTimestamp()
    try
        beginTransaction()
        // Assuming 'doc_request_note' is a dedicated field or stored in a generic 'status_note' column
        update_sql ← "UPDATE D3_QUERIES SET status = ?, doc_request_note = ?, updated_at = ? WHERE query_id = ?"
        executeParameterizedUpdate(update_sql, ["Documents Required", doc_note, current_time, query_id])
        audit_sql ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id, timestamp) VALUES (?, ?, ?, ?, ?)"
        executeParameterizedInsert(audit_sql, [tic_id, "Request Document", "Query", query_id, current_time])
        commitTransaction()
    catch DatabaseException as db_err
        rollbackTransaction()
        logError(db_err)
        return error "Database Error: Failed to update query state and commit audit log"
    end try
    // 5. Fire-and-Forget Asynchronous Notification
    try
        call DispatchNotificationAsync(target_query.student_id, "Document Required", query_id)
    catch NetworkException as net_err
        logError("Warning: Failed to dispatch document request notification to student for query " + query_id)
    end try
    return success "Document request successfully logged and student notified", query_id
```

71: **end procedure**

Use Case 15: Review Submitted Documents

1: **procedure** TICReview**Do**cument(*tic_id, query_id, **do**c_id, review_decision, review_notes*)
2: *// 1. Initial Type and Null Validation*
3: **if** tic_id is null or query_id is null or doc_id is null or review_decision is null
4: **then**
5: **return** error *"Validation Failed: Core parameters cannot be null"*
6: **end if**
7: **if** typeof(*tic_id*) != "*string*" or typeof(*query_id*) != "*string*" or typeof(*doc_id*) != "*string*" or typeof(*review_decision*) != "*string*"
8: **then**
9: **return** error *"Validation Failed: Invalid parameter data types"*
10: **end if**
11: *// 2. Explicit Decision Validation Loop*
12: valid_decisions ← array of strings {"Accept", "Reject"}
13: is_valid_decision ← false
14: **for** i ← 0 to length(*valid_decisions*) - 1
15: **do**
16: **if** review_decision = valid_decisions[*i*]
17: **then**
18: is_valid_decision ← true
19: **break**
20: **end if**
21: **end for**
22: **if** is_valid_decision = false
23: **then**
24: **return** error *"Validation Failed: Decision must be 'Accept' or 'Reject' "*
25: **end if**
26: *// Note constraint: Rejections require a justification*
27: **if** review_decision = "*Reject*" and (review_notes is null or length(review_notes) < 5)
28: **then**
29: **return** error *"Validation Failed: Rejection requires detailed review notes"*
30: **end if**
31: *// 3. Strict Role-Based Access Control (RBAC) Validation*
32: **try**
33: rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id = R.role_id WHERE U.user_id = ?"
34: role_record ← executeParameterizedQuery(*rbac_query, [tic_id]*)
35: **if** role_record is empty or role_record.role_name != "*TIC*"
36: **then**
37: **return** error *"Security Error: Unauthorized access. TIC privileges required."*
38: **end if**
39: **catch** DatabaseException as db_err
40: logError(*db_err*)

```
41:     return error "System Error: Unable to validate permissions"
42:   end try
43:   // 4. Explicit Ownership and State Validation (IDOR Prevention)
44:   try
45:     // Verify Query Ownership
46:     query_check ← "SELECT assigned_tic, student_id FROM D3_QUERIES WHERE query_id =
?"
47:     target_query ← executeParameterizedQuery(query_check, [query_id])
48:     if target_query is empty
49:     then
50:       return error "Database Error: Target query reference not found"
51:     end if
52:     if target_query.assigned_tic != tic_id
53:     then
54:       return error "Security Error: Unauthorized. You can only review documents for queries
assigned to you."
55:     end if
56:     // Verify Document Exists, belongs to this Query, and is Pending
57:     doc_check ← "SELECT status FROM D4_DOCUMENTS WHERE document_id = ? AND
query_id = ?"
58:     target_doc ← executeParameterizedQuery(doc_check, [doc_id, query_id])
59:     if target_doc is empty
60:     then
61:       return error "Database Error: Document not found or does not belong to the specified query"
62:     end if
63:     if target_doc.status != "Pending Review"
64:     then
65:       return error "State Error: Document is not in a reviewable state"
66:     end if
67:   catch DatabaseException as db_err
68:     logError(db_err)
69:     return error "System Error: Unable to validate query and document integrity"
70:   end try
71:   // 5. Secure Database Transaction for Dual-State Transition
72:   current_time ← getSystemTimestamp()
73:   notification_type ← ""
74:   try
75:     beginTransaction()
76:     if review_decision = "Accept"
77:     then
78:       doc_update ← "UPDATE D4_DOCUMENTS SET status = ?, review_notes = ?, updated_at =
? WHERE document_id = ?"
79:       executeParameterizedUpdate(doc_update, ["Verified", review_notes, current_time, doc_id])
80:       query_update ← "UPDATE D3_QUERIES SET status = ?, updated_at = ? WHERE query_id
= ?"
81:       executeParameterizedUpdate(query_update, ["Assigned to TIC", current_time, query_id])
82:       notification_type ← "Document Accepted"
83:     else if review_decision = "Reject"
84:     then
```

```
85:         doc_update ← "UPDATE D4_DOCUMENTS SET status = ?, review_notes = ?, updated_at =
? WHERE document_id = ?"
86:         executeParameterizedUpdate(doc_update, ["Rejected", review_notes, current_time, doc_id])
87:         query_update ← "UPDATE D3_QUERIES SET status = ?, updated_at = ? WHERE query_id
= ?"
88:         executeParameterizedUpdate(query_update, ["Documents Required", current_time,
query_id])
89:         notification_type ← "Document Rejected"
90:       end if
91:       audit_sql ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
92:       executeParameterizedInsert(audit_sql, [tic_id, concat("Review Document - ", review_decision),
"Document", doc_id, current_time])
93:       commitTransaction()
94:    catch DatabaseException as db_err
95:       rollbackTransaction()
96:       logError(db_err)
97:       return error "Database Error: Failed to update document and query states"
98:    end try
99:    // 6. Fire-and-Forget Asynchronous Notification
100:   try
101:      call DispatchNotificationAsync(target_query.student_id, notification_type, query_id)
102:   catch NetworkException as net_err
103:      logError("Warning: Failed to dispatch document review notification to student for query " +
query_id)
104:   end try
105:   return success concat("Document successfully ", review_decision, "ed"), doc_id
106: end procedure
```

Use Case 16: View timetable

```
1: procedure ViewTimetableAvailability(tic_id, target_date)
2:    // 1. Initial Type, Null, and Boundary Validation
3:    if tic_id is null or target_date is null
4:    then
5:       return error "Validation Failed: Parameters cannot be null"
6:    end if
7:    if typeof(tic_id) != "string" or typeof(target_date) != "string"
8:    then
9:       return error "Validation Failed: Invalid parameter data types"
10:   end if
11:   // Explicitly validate ISO 8601 Date Format (YYYY-MM-DD) Length
12:   if length(target_date) != 10
13:   then
14:      return error "Validation Failed: Invalid date format, expected YYYY-MM-DD"
15:   end if
```

```
16:   // 2. Strict Role-Based Access Control (RBAC) Validation
17:   try
18:       rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
19:       role_record ← executeParameterizedQuery(rbac_query, [tic_id])
20:       if role_record is empty or role_record.role_name != "TIC"
21:       then
22:           return error "Security Error: Unauthorized access. TIC privileges required."
23:       end if
24:   catch DatabaseException as db_err
25:       logError(db_err)
26:       return error "System Error: Unable to validate permissions"
27:   end try
28:   // 3. Resilient External Timetable Fetch
29:   base_schedule ← empty list
30:   try
31:       // Returns an array of time slots, e.g., [{start_time: "09:00", end_time: "10:00"}, ...]
32:       base_schedule ← call ExternalTimetableService_Fetch(tic_id, target_date)
33:       if base_schedule is null
34:       then
35:           return error "API Error: Timetable service returned a null schedule"
36:       end if
37:   catch NetworkException as net_err
38:       logError(net_err)
39:       return error "System Error: External timetable service is temporarily unavailable"
40:   end try
41:   // If the base schedule is empty (e.g., weekend/holiday), return early
42:   if length(base_schedule) = 0
43:   then
44:       return success "No slots available on this date", empty list
45:   end if
46:   // 4. Secure Database Retrieval of Existing Bookings
47:   booked_meetings ← empty list
48:   try
49:       // Fetch only active meetings to prevent cancelled ones from blocking slots
50:       meeting_query ← "SELECT start_time, end_time FROM D5_MEETINGS WHERE
assigned_tic = ? AND meeting_date = ? AND status != ?"
51:       booked_meetings ← executeParameterizedQuery(meeting_query, [tic_id, target_date,
"Cancelled"])
52:   catch DatabaseException as db_err
53:       logError(db_err)
54:       return error "Database Error: Failed to retrieve existing meeting bookings"
55:   end try
56:   // 5. Explicit Algorithmic Conflict Resolution (Nested Loops)
57:   available_slots ← empty list
58:   for i ← 0 to length(base_schedule) - 1
59:   do
60:       current_slot ← base_schedule[i]
61:       has_conflict ← false
62:       for j ← 0 to length(booked_meetings) - 1
```

```
63:       do
64:          booked_slot ← booked_meetings[j]
65:          // Explicit Time Overlap: Two intervals overlap if (Start1 < End2) AND (End1 > Start2)
66:          if (current_slot.start_time < booked_slot.end_time) and (current_slot.end_time >
booked_slot.start_time)
67:          then
68:             has_conflict ← true
69:             break // Optimization: Exit inner loop immediately upon finding first conflict
70:          end if
71:       end for
72:       // If the inner loop finishes without tripping the conflict flag, the slot is free
73:       if has_conflict = false
74:       then
75:          insert current_slot into available_slots
76:       end if
77:    end for
78:    return success "Timetable availability calculated successfully", available_slots
79: end procedure
```

Use Case 17: Schedule/Offer/Cancel meetings

```
1: procedure  ManageMeeting(tic_id, action_type, meeting_data)
2:    // 1. Initial Type and Null Validation
3:    if tic_id is null or action_type is null or meeting_data is null
4:    then
5:       return error "Validation Failed: Core parameters cannot be null"
6:    end if
7:    if typeof(tic_id) != "string" or typeof(action_type) != "string" or typeof(meeting_data) !=
"dictionary"
8:    then
9:       return error "Validation Failed: Invalid parameter data types"
10:   end if
11:   // 2. Explicit Action Type Validation Loop
12:   valid_actions ← array of strings {"Schedule", "Cancel"}
13:   is_valid_action ← false
14:   for i ← 0 to length(valid_actions) - 1
15:   do
16:      if action_type = valid_actions[i]
17:      then
18:         is_valid_action ← true
19:         break
20:      end if
21:   end for
22:   if is_valid_action = false
23:   then
24:      return error "Validation Failed: Action must be 'Schedule' or 'Cancel'"
```

```
25:   end if
26:   // 3. Strict Role-Based Access Control (RBAC) Validation
27:   try
28:       rbac_query ← "SELECT R.role_name FROM USERS U JOIN ROLE R ON U.role_id =
R.role_id WHERE U.user_id = ?"
29:       role_record ← executeParameterizedQuery(rbac_query, [tic_id])
30:       if role_record is empty or role_record.role_name != "TIC"
31:       then
32:           return error "Security Error: Unauthorized access. TIC privileges required."
33:       end if
34:   catch DatabaseException as db_err
35:       logError(db_err)
36:       return error "System Error: Unable to validate permissions"
37:   end try
38:   current_time ← getSystemTimestamp()
39:   student_id_target ← null
40:   reference_id ← null
41:   notification_type ← ""
42:   // 4. Branch A: Schedule Meeting
43:   if action_type = "Schedule"
44:   then
45:       if meeting_data["query_id"] is null or meeting_data["date"] is null or
meeting_data["start_time"] is null or meeting_data["end_time"] is null
46:       then
47:           return error "Validation Failed: Incomplete meeting scheduling data"
48:       end if
49:       try
50:           // Verify Query Ownership (IDOR Prevention)
51:           query_check ← "SELECT assigned_tic, student_id FROM D3_QUERIES WHERE query_id
= ?"
52:           target_query ← executeParameterizedQuery(query_check, [meeting_data["query_id"]])
53:           if target_query is empty or target_query.assigned_tic != tic_id
54:           then
55:               return error "Security Error: Cannot schedule meeting for an unassigned or invalid
query"
56:           end if
57:           student_id_target ← target_query.student_id
58:           // Concurrency Control: Double-Booking Prevention at Database Level
59:           // Overlap: (existing_start < new_end) AND (existing_end > new_start)
60:           overlap_check ← "SELECT meeting_id FROM D5_MEETINGS WHERE assigned_tic = ?
AND meeting_date = ? AND status != ? AND start_time < ? AND end_time > ?"
61:           conflict_record ← executeParameterizedQuery(overlap_check, [tic_id, meeting_data["date"],
"Cancelled", meeting_data["end_time"], meeting_data["start_time"]])
62:           if conflict_record is not empty
63:           then
64:               return error "Concurrency Error: Time slot has already been booked (Race Condition
Prevented)"
65:           end if
66:           // Transaction: Insert Meeting and Audit
67:           reference_id ← generateSecureUUID()
```

```
68:         beginTransaction()
69:         insert_meeting ← "INSERT INTO D5_MEETINGS (meeting_id, query_id, assigned_tic,
student_id, meeting_date, start_time, end_time, status, created_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"
70:         executeParameterizedInsert(insert_meeting, [reference_id, meeting_data["query_id"], tic_id,
student_id_target, meeting_data["date"], meeting_data["start_time"], meeting_data["end_time"],
"Scheduled", current_time])
71:         insert_audit ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
72:         executeParameterizedInsert(insert_audit, [tic_id, "Schedule Meeting", "Meeting",
reference_id, current_time])
73:         commitTransaction()
74:         notification_type ← "Meeting Scheduled"
75:     catch DatabaseException as db_err
76:         rollbackTransaction()
77:         logError(db_err)
78:         return error "Database Error: Failed to schedule meeting safely"
79:     end try
80:  // 5. Branch B: Cancel Meeting
81:  else if action_type = "Cancel"
82:  then
83:     if meeting_data["meeting_id"] is null
84:     then
85:         return error "Validation Failed: Meeting ID required for cancellation"
86:     end if
87:     reference_id ← meeting_data["meeting_id"]
88:     try
89:         // Verify Meeting Ownership and State (IDOR Prevention)
90:         meeting_check ← "SELECT assigned_tic, student_id, status FROM D5_MEETINGS
WHERE meeting_id = ?"
91:         target_meeting ← executeParameterizedQuery(meeting_check, [reference_id])
92:         if target_meeting is empty
93:         then
94:             return error "Database Error: Meeting reference not found"
95:         end if
96:         if target_meeting.assigned_tic != tic_id
97:         then
98:             return error "Security Error: Unauthorized attempt to cancel another TIC's meeting"
99:         end if
100:        if target_meeting.status = "Cancelled"
101:        then
102:            return error "State Error: Meeting is already cancelled"
103:        end if
104:        student_id_target ← target_meeting.student_id
105:        // Transaction: Update Meeting and Audit
106:        beginTransaction()
107:        cancel_update ← "UPDATE D5_MEETINGS SET status = ?, updated_at = ? WHERE
meeting_id = ?"
108:        executeParameterizedUpdate(cancel_update, ["Cancelled", current_time, reference_id])
109:        insert_audit ← "INSERT INTO D7_AUDIT_LOGS (user_id, action_type, entity, entity_id,
timestamp) VALUES (?, ?, ?, ?, ?)"
```

```
110:         executeParameterizedInsert(insert_audit, [tic_id, "Cancel Meeting", "Meeting", reference_id, current_time])
111:         commitTransaction()
112:         notification_type ← "Meeting Cancelled"
113:     catch DatabaseException as db_err
114:         rollbackTransaction()
115:         logError(db_err)
116:         return error "Database Error: Failed to cancel meeting safely"
117:     end try
118:   end if
119:   // 6. Fire-and-Forget Asynchronous Notification
120:   try
121:     call DispatchNotificationAsync(student_id_target, notification_type, reference_id)
122:   catch NetworkException as net_err
123:     logError("Warning: Failed to dispatch meeting notification to student for meeting " + reference_id)
124:   end try
125:   return success concat("Meeting successfully ", action_type, "d"), reference_id
126: end procedure
```